

Array — 데이터 저장 + 처리를 모두 담당하는 핵심 자료구조

JS에서 Array는 Java의 ArrayList + Stream 역할을 동시에 수행한다. 코딩테스트의 거의 모든 문제에서 사용.

위치: JavaScript의 가장 기본적인 자료구조. 순서가 있고, 중복 허용, 동적 크기.

핵심 메서드

메서드	설명	원본 변경	시간복잡도
<code>push(e)</code>	끝에 추가	O	O(1)
<code>pop()</code>	끝에서 제거 후 반환	O	O(1)
<code>shift()</code>	앞에서 제거 후 반환	O	O(n)
<code>unshift(e)</code>	앞에 추가	O	O(n)
<code>splice(start, count, ...items)</code>	특정 위치에서 제거/추가	O	O(n)
<code>slice(start, end)</code>	부분 배열 복사	X	O(n)
<code>includes(e)</code>	값 존재 여부 (boolean)	X	O(n)
<code>indexOf(e)</code>	값의 첫 번째 인덱스 (-1 if 없음)	X	O(n)
<code>join(separator)</code>	배열 → 문자열	X	O(n)
<code>length</code>	배열 길이 (속성)	-	O(1)
<code>concat(arr)</code>	배열 합치기	X	O(n)
<code>fill(value, start, end)</code>	특정 값으로 채우기	O	O(n)

패턴

코딩테스트 출제 빈도: 패턴1 스택처럼 사용 > 패턴2 부분 배열 추출 > 패턴3 존재 여부 + 위치 확인 (JS: 찾아서 제거) > 패턴4 모든 쌍 비교 (JS: 2차원 배열 순회)

패턴 1: 스택처럼 사용 — push / pop

언제 쓰는가

- 괄호 검증, 이전 값과 비교, LIFO(후입선출) 패턴

실무에서는

- Undo/Redo 기능, 내비게이션 히스토리

```
// 괄호 유효성 검사
function isValidParentheses(s) {
  const stack = [];
  const pairs = { ')': '(', ']': '[', '}': '{' };

  for (const char of s) {
    if ('([{'.includes(char)) {
      stack.push(char); // 여는 괄호면 push
    } else {
      if (stack.pop() !== pairs[char]) return false; // 닫는 괄호면 pop
    }
  }

  return stack.length === 0;
}

console.log(isValidParentheses("({[]})")); // true
console.log(isValidParentheses("({[])")); // false
console.log(isValidParentheses("(")); // false
```

패턴 2: 부분 배열 추출 — slice

언제 쓰는가

- 원본을 건드리지 않고 특정 구간을 추출할 때

실무에서는

- 페이지네이션, 무한 스크롤 데이터 슬라이싱

```
const arr = [10, 20, 30, 40, 50];

// 인덱스 1~3 추출 (end 미포함)
const sub = arr.slice(1, 4);
console.log(sub); // [20, 30, 40]
console.log(arr); // [10, 20, 30, 40, 50] - 원본 그대로

// 뒤에서 2개
const last2 = arr.slice(-2);
console.log(last2); // [40, 50]

// 배열 복사 (얕은 복사)
const copy = arr.slice();
console.log(copy); // [10, 20, 30, 40, 50]

// 페이지네이션 예시
function getPage(arr, page, size) {
  const start = (page - 1) * size;
```

```

    return arr.slice(start, start + size);
}

const items = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log(getPage(items, 1, 3)); // [1, 2, 3]
console.log(getPage(items, 2, 3)); // [4, 5, 6]
console.log(getPage(items, 4, 3)); // [10]

```

패턴 3: 찾아서 제거 — indexOf + splice

언제 쓰는가

- 특정 값을 찾아서 삭제할 때

실무에서는

- 장바구니 아이템 삭제, 태그 제거

```

const cart = ["apple", "banana", "cherry", "banana"];

// "banana" 찾아서 첫 번째만 제거
const idx = cart.indexOf("banana");
if (idx !== -1) {
    cart.splice(idx, 1); // 인덱스 idx에서 1개 제거
}
console.log(cart); // ["apple", "cherry", "banana"]

// 특정 값 모두 제거
let tags = ["js", "python", "js", "java", "js"];
while (tags.indexOf("js") !== -1) {
    tags.splice(tags.indexOf("js"), 1);
}
console.log(tags); // ["python", "java"]

// 더 간결한 방법: filter 사용 (원본 유지)
const original = ["js", "python", "js", "java", "js"];
const filtered = original.filter(tag => tag !== "js");
console.log(filtered); // ["python", "java"]

```

패턴 4: 2차원 배열 순회 — 이중 인덱스

언제 쓰는가

- 행렬(matrix), 격자(grid) 데이터 처리
- BFS/DFS에서 상하좌우 탐색

실무에서는

- 스프레드시트 데이터 처리, 게임 보드

```
// 3x3 행렬 생성 및 순회
const matrix = [
  [1, 2, 3],
  [4, 5, 6],
  [7, 8, 9]
];

// 전체 순회
for (let r = 0; r < matrix.length; r++) {
  for (let c = 0; c < matrix[0].length; c++) {
    process.stdout.write(matrix[r][c] + " ");
  }
}
console.log(); // 1 2 3 4 5 6 7 8 9

// 상하좌우 방향 탐색 (BFS/DFS에서 필수)
const dr = [-1, 1, 0, 0]; // 상, 하, 좌, 우
const dc = [0, 0, -1, 1];

function getNeighbors(r, c, rows, cols) {
  const neighbors = [];
  for (let d = 0; d < 4; d++) {
    const nr = r + dr[d];
    const nc = c + dc[d];
    if (nr >= 0 && nr < rows && nc >= 0 && nc < cols) {
      neighbors.push([nr, nc]);
    }
  }
  return neighbors;
}

// (1, 1) = 5의 상하좌우 이웃
console.log(getNeighbors(1, 1, 3, 3)); // [[0,1], [2,1], [1,0], [1,2]]

// 2차원 배열 초기화 (주의: fill로 하면 참조 공유 문제)
// 올바른 방법
const grid = Array.from({ length: 3 }, () => new Array(3).fill(0));
grid[0][0] = 1;
console.log(grid); // [[1,0,0], [0,0,0], [0,0,0]] - 올바른

// 잘못된 방법
// const bad = new Array(3).fill(new Array(3).fill(0));
// bad[0][0] = 1;
// console.log(bad); // [[1,0,0], [1,0,0], [1,0,0]] - 모든 행이 같은 배열 참조!
```

주의사항

splice는 원본 변경, slice는 원본 보존 — 반드시 구분

```
const a = [1, 2, 3, 4, 5];  
a.splice(1, 2); // 인덱스 1에서 2개 제거 → a = [1, 4, 5] (원본 변경!)  
  
const b = [1, 2, 3, 4, 5];  
const c = b.slice(1, 3); // c = [2, 3], b는 그대로 (원본 보존)
```

length를 직접 수정하면 배열이 잘린다

```
const arr = [1, 2, 3, 4, 5];  
arr.length = 3;  
console.log(arr); // [1, 2, 3] - 뒤쪽 요소 삭제됨!
```

빈 배열 비교: [] === [] 는 false

```
console.log([] === []); // false - 서로 다른 참조(객체)  
console.log([1] === [1]); // false - 마찬가지로  
  
// 배열 내용 비교는 JSON.stringify 또는 every 사용  
const x = [1, 2, 3];  
const y = [1, 2, 3];  
console.log(JSON.stringify(x) === JSON.stringify(y)); // true
```